NCML project Report

TABLE OF CONTENTS

(Drowsiness and Distraction Detection using Computer Vision)

1. Face Detection
2. Feature Extraction
   - Eye Aspect Ratio (EAR)
   - Mouth Aspect Ratio (MAR)
   - Gaze Tracking
   - Head Pose Estimation
3. Temporal Analysis
4. Classification Logic
5. Alert Mechanism
   - Main Application
   - Configuration
   - Detection Algorithm
6. Code
7. Output

DRIVER AI MONITORING SYSTEM

(Drowsiness and Distraction Detection using Computer Vision)

ABSTRACT

Driver fatigue and distraction are major contributors to road accidents globally, leading to reduced reaction time and impaired decision-making. This project presents a real-time **Driver AI Monitoring System** designed to detect drowsiness and distraction using computer vision techniques.

The system leverages **MediaPipe FaceMesh** for facial landmark detection and computes behavioral metrics such as **Eye Aspect Ratio (EAR)**, **Mouth Aspect Ratio (MAR)**, **gaze ratio**, and **head pose estimation**. These features are analyzed using temporal logic and dynamic thresholds to accurately classify driver states.

A modern **Streamlit-based dashboard** provides real-time visualization, status monitoring, and fatigue analysis. When unsafe conditions such as drowsiness or distraction are detected, an alert is triggered using an audio alarm system.

This system demonstrates the integration of artificial intelligence, real-time video processing, and human behavioral analysis to enhance road safety.

INTRODUCTION

Road safety has become a critical concern due to the increasing number of accidents caused by human error. Among these, driver fatigue and distraction are significant factors that reduce awareness and increase the likelihood of collisions.

Traditional monitoring systems are often limited in scope and fail to provide real-time feedback. With advancements in computer vision and AI, it is now possible to develop intelligent systems capable of continuously analyzing driver behavior.

This project introduces a **real-time monitoring system** that:

- Continuously tracks driver facial features
- Detects signs of fatigue and distraction
- Provides instant feedback through alerts
- Visualizes driver state using an interactive dashboard

The system aims to bridge the gap between human limitations and intelligent automation.

PROBLEM STATEMENT

Driver drowsiness and distraction significantly increase accident risks due to delayed responses and reduced situational awareness. Existing solutions suffer from several limitations:

- High cost and lack of accessibility

- Limited detection (single-factor systems)
- Lack of real-time feedback
- Poor adaptability to different users

The objective is to develop a **real-time, cost-effective, and multi-feature detection system** that:

- Monitors multiple behavioral indicators
- Detects unsafe conditions accurately
- Provides immediate alerts
- Works efficiently under varying conditions

OBJECTIVES

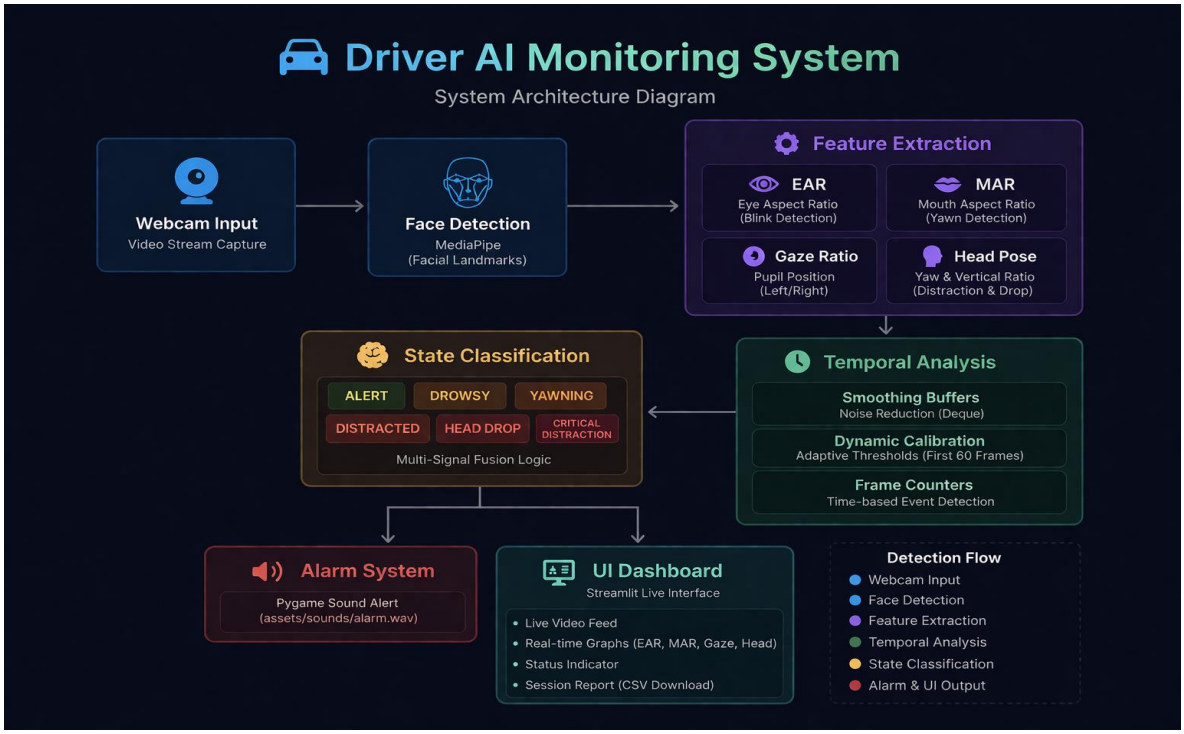
The primary objectives of this project are:

- To detect eye closure using Eye Aspect Ratio (EAR)
- To detect yawning using Mouth Aspect Ratio (MAR)
- To track gaze direction for distraction analysis
- To analyze head movement for posture detection
- To implement an alarm system for unsafe conditions
- To develop a real-time dashboard for monitoring and analytics

---

SYSTEM ARCHITECTURE

The system follows a structured pipeline:



METHODOLOGY

1. Face Detection

The system uses **MediaPipe FaceMesh** to detect 468 facial landmarks in real time, enabling accurate feature extraction.

---

2. Feature Extraction

Eye Aspect Ratio (EAR)

Measures eye openness to detect blinking and prolonged closure (drowsiness).

Mouth Aspect Ratio (MAR)

Detects yawning behavior by analyzing mouth opening.

Gaze Tracking

Tracks pupil movement to determine whether the driver is looking away from the road.

Head Pose Estimation

Detects head tilt and rotation to identify distraction or fatigue.

---

3. Temporal Analysis

To improve reliability:

- Frame counters track duration of events
- Smoothing buffers reduce noise
- Calibration phase adapts thresholds per user

---

4. Classification Logic

Driver states include:

- ALERT
- SEMI-DROWSY / SEMI-YAWNING
- DROWSY
- YAWNING
- DISTRACTED
- HEAD DROP
- CRITICAL DISTRACTION
-

---

5. Alert Mechanism

An alarm is triggered when unsafe conditions persist for a defined duration using **pygame audio system**.

IMPLEMENTATION

Main Application

The system interface is built using **Streamlit**, providing:

- Live webcam feed
- Real-time metric visualization
- Fatigue score tracking
- Session analytics

Configuration

The system parameters include:

- EAR and MAR thresholds
- Frame counters for detection
- Gaze and head thresholds
- Calibration settings

Detection Algorithm

The detection system uses:

- Dynamic threshold adjustment
- Multi-signal fusion
- Time-based decision making
- Noise reduction using buffers

---

code

```
import streamlit as st
import cv2
import time
import pandas as pd
import numpy as np
from src.detector import DrowsinessDetector

# ========================================
# 1. PAGE CONFIG & SESSION STATE
# ========================================
st.set_page_config(page_title="Driver AI Dashboard", layout="wide", initial_sidebar_state="expanded")
for k in ["ear_list", "mar_list", "gaze_list", "head_list", "drive_history"]:
    if k not in st.session_state: st.session_state[k] = []
if "fatigue_score" not in st.session_state: st.session_state.fatigue_score = 0
if "cap" not in st.session_state: st.session_state.cap = None
# ========================================
# 2. THEME & DARK BUTTON CSS
# ========================================
st.markdown("""
<style>
    .stApp { background-color: #020617; color: #f8fafc; }
    .dashboard-header { background: rgba(30, 41, 59, 0.4); border: 1px solid rgba(255, 255, 255, 0.05); border-radius: 16px; padding: 25px; margin-bottom:
25px; text-align: center; backdrop-filter: blur(12px); }
    .dashboard-header h1 { color: #38bdf8; margin: 0; font-size: 2rem; font-weight: 800; }
    .dashboard-header p { color: #94a3b8; font-size: 0.9rem; margin-top: 5px; text-transform: uppercase; letter-spacing: 2px; }
    .glass-card { background: rgba(30, 41, 59, 0.5); border: 1px solid rgba(255, 255, 255, 0.1); border-radius: 16px; padding: 20px; margin-bottom: 15px;
backdrop-filter: blur(12px); text-align: center; }
    .card-label { color: #94a3b8; font-size: 0.75rem; font-weight: 600; text-transform: uppercase; letter-spacing: 1.5px; margin-bottom: 8px; }
    .card-value { color: #f8fafc; font-size: 2.2rem; font-weight: 700; margin: 0; line-height: 1; }

    .stButton>button, .stDownloadButton>button {
        background-color: #0f172a !important; color: #f8fafc !important;
        border: 1px solid #334155 !important; border-radius: 12px !important;
        padding: 10px 24px !important; width: 100% !important; transition: 0.3s !important;
    }
    .stButton>button:hover, .stDownloadButton>button:hover {
        background-color: #38bdf8 !important; color: #020617 !important; border: 1px solid #38bdf8 !important;
    }
    .status-text { font-size: clamp(1.4rem, 1.8vw, 2.2rem); font-weight: 900; margin: 0; }
    .color-alert { color: #10b981; } .color-warn { color: #facc15; } .color-danger { color: #ef4444; } .color-calib { color: #38bdf8; }
    .blink { animation: pulse 1s infinite; }
    @keyframes pulse { 0%, 100% { opacity: 1; } 50% { opacity: 0.4; } }
    .section-label { font-size: 0.85rem; color: #cbd5e1; font-weight: 600; margin-bottom: 10px; display: block; text-transform: uppercase; }
</style>
""", unsafe_allow_html=True)
```

```
st.markdown('<div class="dashboard-header"><h1>□ Driver AI Monitoring System</h1><p>Drowsiness, Fatigue, and Multi-Axis Distraction Detection</p></div>', unsafe_allow_html=True)
# ==========================================
# 3. LAYOUT
# ==========================================
run = st.sidebar.checkbox("□ Start Monitoring", value=False)
if st.sidebar.button("□ Reset Session"):
    for k in ["ear_list", "mar_list", "gaze_list", "head_list", "drive_history"]: st.session_state[k] = []
    st.session_state.fatigue_score = 0; st.rerun()
col_main, col_dash = st.columns([2.8, 1.2], gap="large")
with col_main:
    main_placeholder = st.empty()
    r1c1, r1c2 = st.columns(2); r2c1, r2c2 = st.columns(2)
    with r1c1: st.markdown('<div class="section-label">□ EAR (Eyes)</div>', unsafe_allow_html=True); ear_g = st.empty()
    with r1c2: st.markdown('<div class="section-label">□ MAR (Mouth)</div>', unsafe_allow_html=True); mar_g = st.empty()
    with r2c1: st.markdown('<div class="section-label">□ GAZE (Pupils)</div>', unsafe_allow_html=True); gaze_g = st.empty()
    with r2c2: st.markdown('<div class="section-label">□ HEAD (Yaw)</div>', unsafe_allow_html=True); head_g = st.empty()
with col_dash:
    status_box = st.empty(); ear_box = st.empty(); mar_box = st.empty(); fatigue_box = st.empty()
def update_dashboard(status, ear, mar, fatigue):
    """
    Updates the UI dashboard cards with the latest status, metrics, and fatigue score.
    """
    # 1. Determine Color, Icon, and Animation based on Status
    if "CALIBRATING" in status:
        color, icon, blink = "color-calib", "□", "blink"
    elif "SUNGLASSES" in status or "GLARE" in status:
        color, icon, blink = "color-calib", "□", ""
    elif status == "ALERT":
        color, icon, blink = "color-alert", "□", ""
    elif status == "CRITICAL DISTRACTION":
        color, icon, blink = "color-danger", "□", "blink"
    elif status in ["DROWSY", "YAWNING", "HEAD DROP", "DISTRACTED"]:
        color, icon, blink = "color-danger", "□", "blink"
    elif "SEMI" in status: # Handles SEMI-DROWSY or SEMI-YAWNING
        color, icon, blink = "color-warn", "□", ""
    else:
        color, icon, blink = "color-warn", "○", ""
    # 2. Render Status Card
    status_box.markdown(f"""
        <div class="glass-card {blink}">
            <div class="card-label">System Status</div>
            <div class="status-text {color}">{icon} {status}</div>
        </div>
    """, unsafe_allow_html=True)
    # 3. Render EAR Card
    ear_box.markdown(f"""
        <div class="glass-card">
            <div class="card-label">Eye Aspect Ratio</div>
            <div class="card-value">{ear:.2f}</div>
        </div>
    """, unsafe_allow_html=True)
    # 4. Render MAR Card
    mar_box.markdown(f"""
        <div class="glass-card">
            <div class="card-label">Mouth Aspect Ratio</div>
            <div class="card-value">{mar:.2f}</div>
        </div>
    """, unsafe_allow_html=True)
```

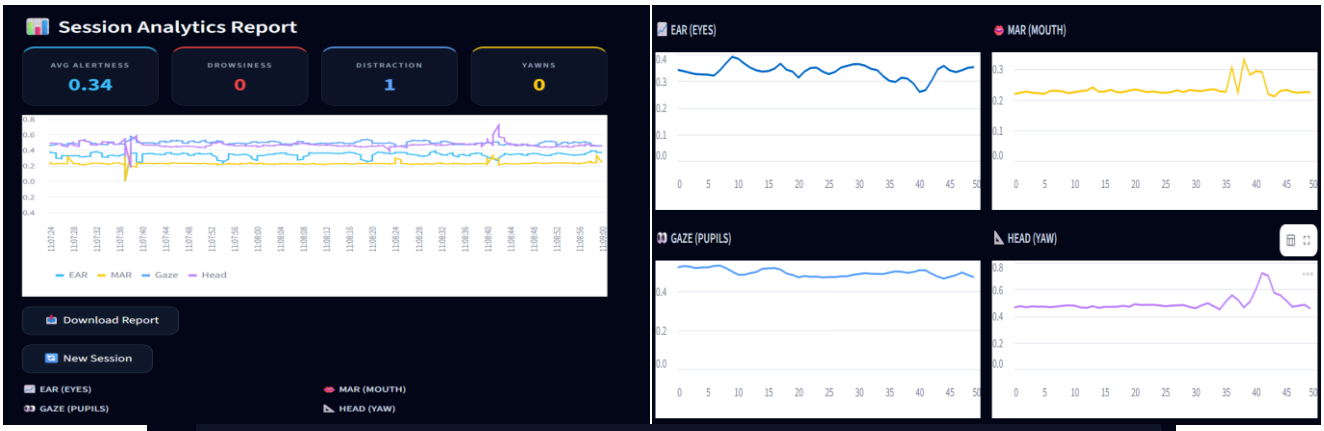
```
    # 5. Render Fatigue Progress Bar
    # Color logic: Green < 30, Yellow < 70, Red >= 70
    bar_color = "#10b981" if fatigue < 30 else "#facc15" if fatigue < 70 else "#ef4444"
    fatigue_box.markdown(f"""
        <div class="glass-card">
            <div class="card-label">Fatigue Level ({fatigue}%)</div>
            <div style="width:100%; height:12px; background:rgba(0,0,0,0.3); border-radius:6px; margin-top:10px; overflow:hidden;">
```

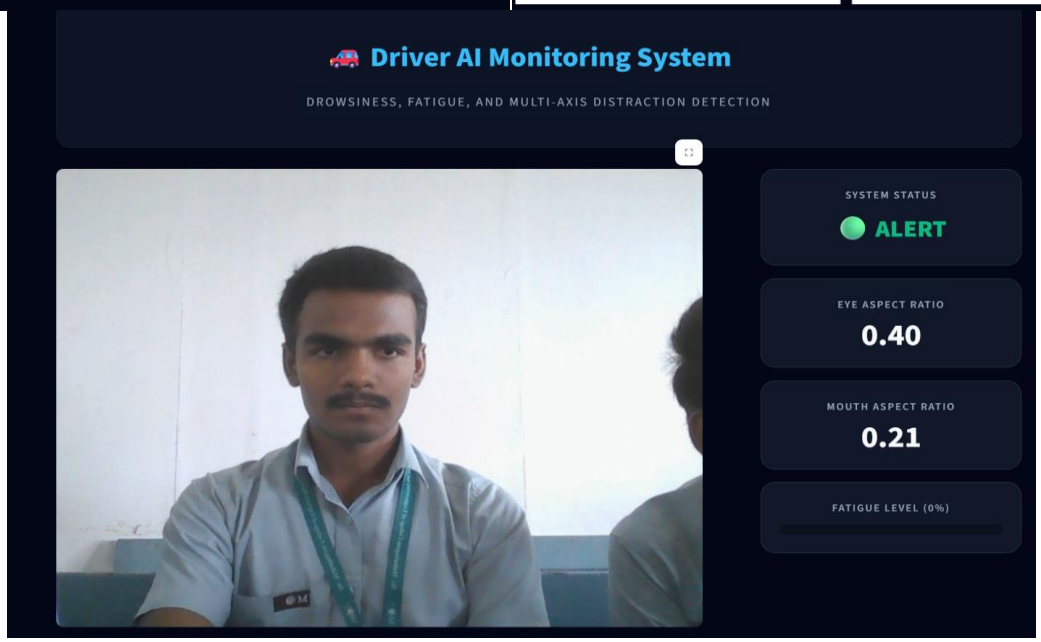
```
            <div style="width:{fatigue}%; height:100%; background:{bar_color}; transition:0.3s; box-shadow:0 0 10px {bar_color};"></div>
        </div>
    </div>
  """, unsafe_allow_html=True)
# ==========================================
# 4. MONITORING
# ==========================================
if run:
  if "detector" not in st.session_state: st.session_state.detector = DrowsinessDetector()
  if st.session_state.cap is None: st.session_state.cap = cv2.VideoCapture(0)
  det, cap = st.session_state.detector, st.session_state.cap

  while run:
    ret, frame = cap.read()
    if not ret: break
    frame = det.process_frame(cv2.flip(frame, 1))
    ear, mar, gaze, head, status = det.last_ear, det.last_mar, det.last_gaze, det.last_head_h, det.last_status
    for l, v in zip([st.session_state.ear_list, st.session_state.mar_list, st.session_state.gaze_list, st.session_state.head_list], [ear, mar, gaze, head]):
      l.append(v)
      if len(l) > 50: l.pop(0)
    if status in ["DROWSY", "YAWNING", "HEAD DROP", "DISTRACTED", "CRITICAL DISTRACTION"]:
      st.session_state.fatigue_score = min(100, st.session_state.fatigue_score + 2)
    else: st.session_state.fatigue_score = max(0, st.session_state.fatigue_score - 1)
    main_placeholder.image(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB), use_container_width=True)
    ear_g.line_chart(st.session_state.ear_list, height=150); mar_g.line_chart(st.session_state.mar_list, height=150, color="#facc15"); gaze_g.line_chart(st.session_state.gaze_list, height=150, color="#60a5fa"); head_g.line_chart(st.session_state.head_list, height=150, color="#c084fc")

    d_status = f"CALIBRATING ({det.calibration_frame_count}/60)" if status == "CALIBRATING" else status
    update_dashboard(d_status, ear, mar, st.session_state.fatigue_score)
      st.session_state.drive_history.append({"Time": time.strftime("%H:%M:%S"), "EAR": ear, "MAR": mar, "Gaze": gaze, "Head": head, "Status": status})
    time.sleep(0.01)
# ==========================================
# 5. REPORT
# ==========================================
else:
  if st.session_state.cap: st.session_state.cap.release(); st.session_state.cap = None
  if st.session_state.drive_history:
    if "detector" in st.session_state: del st.session_state.detector
    df = pd.DataFrame(st.session_state.drive_history)
    with main_placeholder.container():
      st.markdown("### □ Session Analytics Report")
      m1, m2, m3, m4 = st.columns(4)
        stats = [("Avg Alertness", f"{df['EAR'].mean():.2f}", "#38bdf8"), ("Drowsiness", len(df[df['Status'].str.contains('DROWSY', na=False)]), "#ef4444"), ("Distraction", len(df[df['Status'].str.contains('DISTRACT', na=False)]), "#60a5fa"), ("Yawns", len(df[df['Status'].str.contains('YAWN', na=False)]), "#facc15")]
      for col, (l, v, c) in zip([m1, m2, m3, m4], stats):
        col.markdown(f'<div class="glass-card" style="border-top:3px solid {c}; background:rgba(15,23,42,0.8);"><div class="card-label" style="font-size:0.7rem;">{l}</div><div class="card-value" style="color:{c}; font-size:1.8rem;">{v}</div></div>', unsafe_allow_html=True)
      st.line_chart(df.set_index('Time')[['EAR', 'MAR', 'Gaze', 'Head']], color=["#38bdf8", "#facc15", "#60a5fa", "#c084fc"], height=300)
      st.download_button("□ Download Report", df.to_csv(index=False).encode('utf-8'), "report.csv", "text/csv")
      if st.button("□ New Session"): st.session_state.drive_history = []; st.session_state.fatigue_score = 0; st.rerun()
  else:
    update_dashboard("STANDBY", 0, 0, 0)
     main_placeholder.markdown('<div style="height:450px; display:flex; justify-content:center; align-items:center; color:#94a3b8; border:1px dashed #334155; border-radius:16px;">SYSTEM IDLE. ACTIVATE IN SIDEBAR.</div>', unsafe_allow_html=True)
```

OUTPUT

The system produces:

- Live video stream
- Real-time graphs (EAR, MAR, Gaze, Head)
- Driver status indicator
- Alarm alerts
- Downloadable session report





RESULTS

- Successfully detects fatigue and distraction in real time
- Multi-signal approach improves accuracy
- Smooth performance with optimized processing
- Robust under moderate lighting and glasses

ADVANTAGES

- Real-time monitoring
- Multi-feature detection
- Adaptive and scalable
- User-friendly interface
- Cost-effective implementation

LIMITATIONS

- Performance depends on system hardware
- Sensitive to poor lighting conditions
- Streamlit limitations for high FPS processing
- Possible false positives in edge cases

FUTURE SCOPE

- Integration with deep learning models
- Mobile application development
- Cloud-based monitoring systems
- Vehicle integration for automatic control
- Voice-based alert system

CONCLUSION

The Driver AI Monitoring System demonstrates the effective use of computer vision and AI in enhancing road safety. By combining multiple behavioral indicators, the system provides reliable and real-time detection of driver fatigue and distraction.

This project highlights the potential of intelligent monitoring systems and can be further developed into a commercial safety solution.

REFERENCES

- MediaPipe Documentation
- OpenCV Documentation
- Research papers on EAR & MAR
- Python Official Documentation